

2303A52416

batch=35

Assignment =14.5

Task 1 – Image Gallery Website

Create an image gallery web page.

Requirements:

- Display images in a grid layout.
- Use AI to:
 - o Optimize layout using CSS Grid.
 - o Add hover zoom or overlay effects.
 - o Ensure mobile responsiveness.

Code

Image gallery using a list

```
images = ["image1.jpg", "image2.jpg", "image3.jpg", "image4.jpg"]
```

```
print("Image Gallery\n")
```

```
for i, img in enumerate(images, start=1):
```

```
    print(f'{i}. {img}')
```



Explanation

- A **list** stores image file names.
- **enumerate()** displays images with numbers.
- This simulates displaying images in a gallery.

2. Profile Card Generator

Code

Profile card using dictionary

```
profile = {
```

```

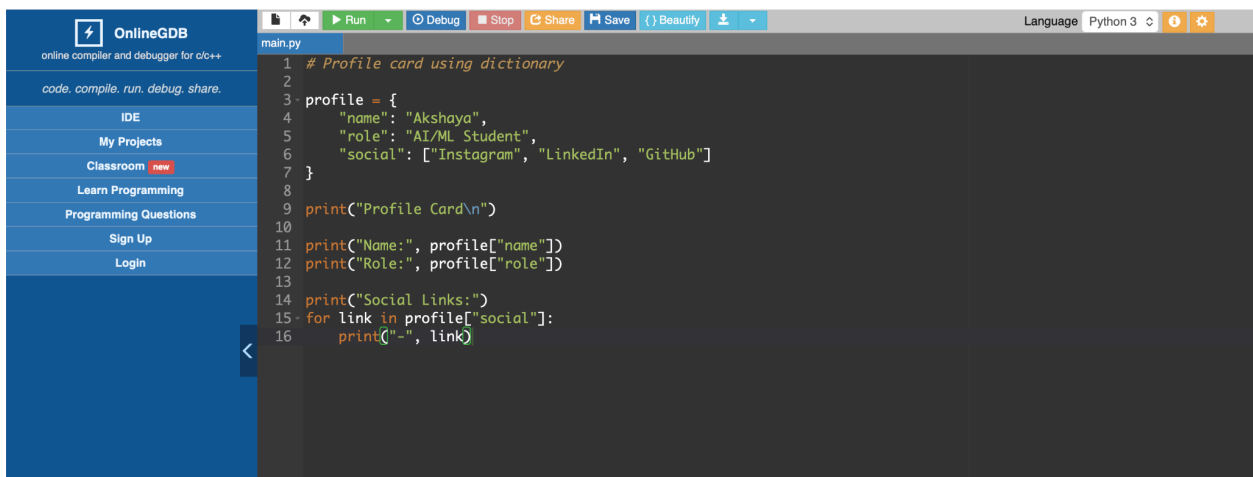
    "name": "Akshaya",
    "role": "AI/ML Student",
    "social": ["Instagram", "LinkedIn", "GitHub"]
}

```

```
print("Profile Card\n")
```

```
print("Name:", profile["name"])
print("Role:", profile["role"])
```

```
print("Social Links:")
for link in profile["social"]:
    print("-", link)
```



The screenshot shows the OnlineGDB IDE interface. On the left is a sidebar with navigation links: IDE, My Projects, Classroom (marked as new), Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays a Python file named 'main.py' with the following code:

```

1  # Profile card using dictionary
2
3  profile = {
4      "name": "Akshaya",
5      "role": "AI/ML Student",
6      "social": ["Instagram", "LinkedIn", "GitHub"]
7  }
8
9  print("Profile Card\n")
10
11 print("Name:", profile["name"])
12 print("Role:", profile["role"])
13
14 print("Social Links:")
15 for link in profile["social"]:
16     print("-", link)

```

Explanation

- A **dictionary** stores profile details.
- Social links are stored in a **list**.
- Loop prints each social media link.

3. Shopping Cart System

Code

Shopping cart system

```
cart = []
```

```
def add_item(price):
    cart.append(price)
```

```
def remove_item(price):
    if price in cart:
        cart.remove(price)
```

```
def total_price():  
    return sum(cart)
```

```
add_item(100)  
add_item(200)  
add_item(150)
```

```
print("Cart Items:", cart)  
print("Total Price:", total_price())
```

The screenshot shows the OnlineGDB web interface. On the left is a sidebar with navigation links: IDE, My Projects, Classroom (marked 'new'), Learn Programming, Programming Questions, Sign Up, and Login. The main area displays a Python file named 'main.py' with the following code:

```
1 # Shopping cart system  
2  
3 cart = []  
4  
5 def add_item(price):  
6     cart.append(price)  
7  
8 def remove_item(price):  
9     if price in cart:  
10        cart.remove(price)  
11  
12 def total_price():  
13     return sum(cart)  
14  
15 add_item(100)  
16 add_item(200)  
17 add_item(150)  
18  
19 print("Cart Items:", cart)  
20 print("Total Price:", total_price())
```

The interface includes a top toolbar with buttons for Run, Debug, Stop, Share, Save, and Beautify, along with a language selector set to Python 3.

Explanation

- `cart` list stores item prices.
- `add_item()` adds products.
- `remove_item()` deletes products.
- `sum()` calculates total cost.

4. Role-Based Access Control

Code

```
# Role based access
```

```
role = input("Enter role (admin/editor/user): ")
```

```
if role == "admin":  
    print("Access: Full system control")
```

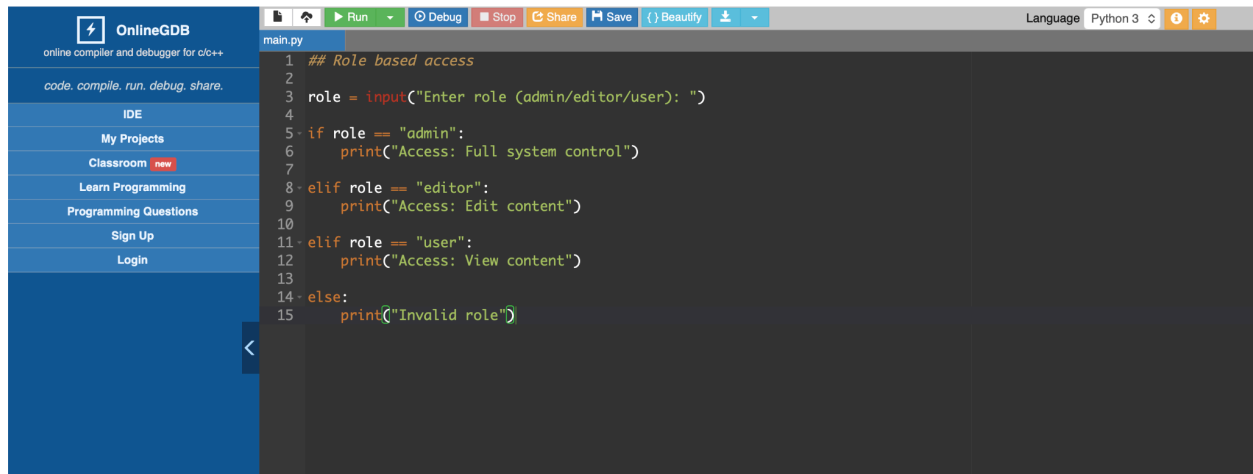
```
elif role == "editor":  
    print("Access: Edit content")
```

```
elif role == "user":
```

```
print("Access: View content")
```

```
else:
```

```
print("Invalid role")
```



The screenshot shows the OnlineGDB web interface. On the left is a sidebar with navigation links: OnlineGDB, code.compile.run.debug.share, IDE, My Projects, Classroom (marked as new), Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays a Python file named 'main.py' with the following code:

```
1  ## Role based access
2
3  role = input("Enter role (admin/editor/user): ")
4
5  if role == "admin":
6      print("Access: Full system control")
7
8  elif role == "editor":
9      print("Access: Edit content")
10
11 elif role == "user":
12     print("Access: View content")
13
14 else:
15     print("Invalid role")
```

Explanation

- User enters a **role**.
- **if-elif** checks access permission.

5. Multi-Language System

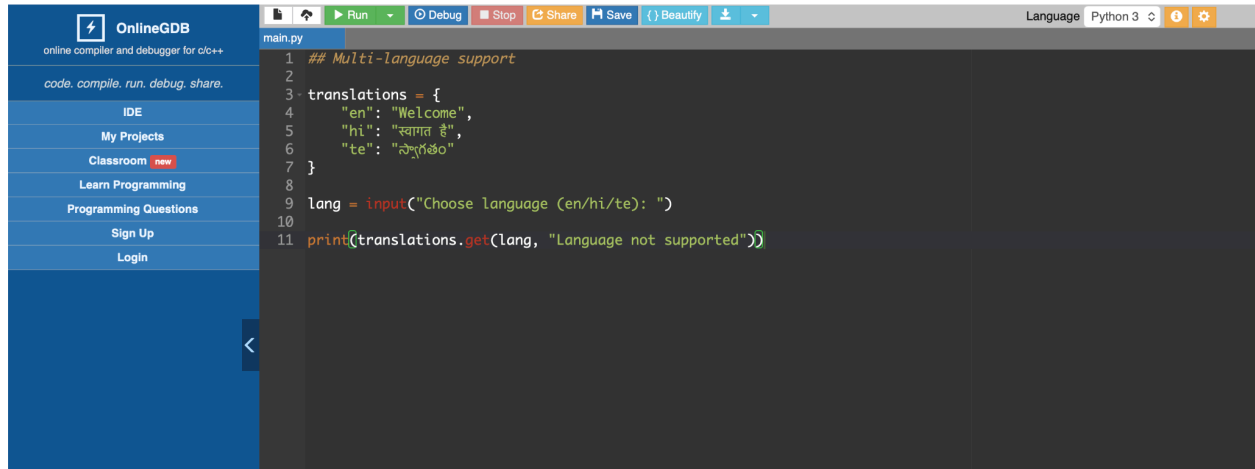
Code

```
# Multi-language support
```

```
translations = {
    "en": "Welcome",
    "hi": "स्वागत है",
    "te": "స్వాగతం"
}
```

```
lang = input("Choose language (en/hi/te): ")
```

```
print(translations.get(lang, "Language not supported"))
```

The image shows a screenshot of the OnlineGDB web interface. On the left is a sidebar with navigation links: IDE, My Projects, Classroom (marked 'new'), Learn Programming, Programming Questions, Sign Up, and Login. The main area displays a Python file named 'main.py' with the following code:

```
1 ## Multi-language support
2
3 translations = {
4     "en": "Welcome",
5     "hi": "स्वागत है",
6     "te": "స్వాగతం"
7 }
8
9 lang = input("Choose language (en/hi/te): ")
10
11 print(translations.get(lang, "Language not supported"))
```

The interface includes a top toolbar with buttons for Run, Debug, Stop, Share, Save, and Beautify. The language is set to Python 3.

Explanation

- Dictionary stores translations.
- `get()` retrieves the selected language text.

6. Online Examination System

Code

import time

```
questions = [  
    ("2 + 2 =", "4"),  
    ("5 * 3 =", "15"),  
    ("10 - 6 =", "4")  
]
```

```
score = 0
```

```
for q, ans in questions:  
    user = input(q + " ")
```

```
    if user == ans:  
        score += 1
```

```
print("Exam finished")  
print("Score:", score)
```



The screenshot shows the OnlineGDB web interface. On the left is a sidebar with navigation links: IDE, My Projects, Classroom (marked 'new'), Learn Programming, Programming Questions, Sign Up, and Login. The main area displays a Python script in a dark-themed editor. The script is a quiz program that imports the 'time' module, defines a list of questions, and uses a loop to ask each question, checking the user's answer and updating the score. The script ends with print statements for 'Exam finished' and the final 'Score'.

```
1 import time
2
3 questions = [
4     ("2 + 2 =", "4"),
5     ("5 * 3 =", "15"),
6     ("10 - 6 =", "4")
7 ]
8
9 score = 0
10
11 for q, ans in questions:
12     user = input(q + " ")
13
14     if user == ans:
15         score += 1
16
17 print("Exam finished")
18 print("Score:", score)
```

Explanation

- Questions stored as **tuples in a list**.
- Loop asks each question.
- Score increases when answer is correct.